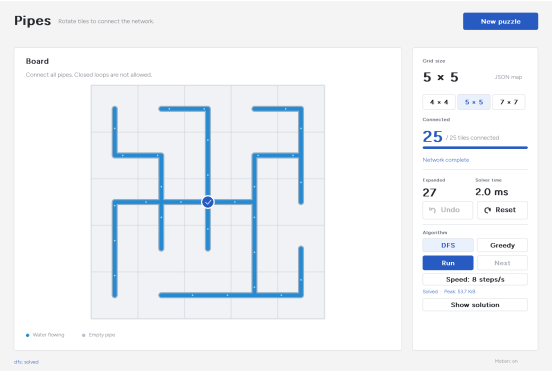
[TÊN TRƯỜNG]
[KHOA / BỘ MÔN]

BÀI TẬP LỚN 1 — TÌM KIẾM

Học kỳ 1, năm học 2026–2027

Môn: Nhập môn Trí tuệ nhân tạo

Pipes & Light Up:
Hiện thực giải thuật tìm kiếm cho hai bài toán Logic Puzzle



Danh sách thành viên

Họ và tên	MSSV	Lớp
[Họ tên 1]	[MSSV 1]	[Lớp 1]
[Họ tên 2]	[MSSV 2]	[Lớp 2]
[Họ tên 3]	[MSSV 3]	[Lớp 3]

Giảng viên hướng dẫn: [Tên giảng viên]

[Địa điểm], ngày 31 tháng 10 năm 2026

Mục lục

1 Mở đầu **2**

1.1 Mục tiêu . . . 2
1.2 Hai bài toán được chọn . . . 2
1.3 Môi trường và công cụ . . . 2
1.4 Tổ chức mã nguồn . . . 3
1.5 Quy trình tìm hiểu và hiện thực . . . 3

2 Bài toán 1: Pipes **3**

2.1 Tìm hiểu đề bài và luật chơi . . . 3
2.2 Phát biểu bài toán tìm kiếm . . . 4
2.3 Không gian tìm kiếm và cắt tỉa dùng chung . . . 4
2.4 Giải thuật tìm kiếm mù: Depth-First Search . . . 5
2.5 Giải thuật heuristic: Greedy Best-First . . . 5
2.6 Hiện thực và demo trực quan . . . 6
2.7 Tạo input . . . 6
2.8 Kết quả thực nghiệm: thời gian và bộ nhớ . . . 6
2.9 Hạn chế . . . 8

3 Bài toán 2: Light Up (Akari) **8**

3.1 Quá trình tìm hiểu và luật chơi . . . 8
3.2 Mô hình hóa bài toán . . . 9
3.3 Thiết kế độ khó và sinh bàn chơi . . . 9
3.4 Lựa chọn và hiện thực giải thuật . . . 10
3.5 Demo và phương pháp thực nghiệm . . . 11
3.6 Kết quả đo . . . 11

4 Kết luận **12**

1 Mở đầu

1.1 Mục tiêu

Bài tập lớn 1 yêu cầu nhóm chọn hai bài toán trong bộ Logic Puzzles tại `https://www.puzzle-pipes.com/` (và các bài toán liên quan trên trang này), sau đó hiện thực bằng Python 3 hai giải thuật tìm kiếm cho **mỗi** bài toán: một giải thuật *tìm kiếm mù* (DFS hoặc BrFS) và một giải thuật *heuristic* (A*, Hill Climbing, v.v.). Bài làm phải có chế độ demo từng bước với hiển thị trực quan dạng game, kèm báo cáo trình bày quá trình tìm hiểu, quá trình hiện thực, bảng số liệu thời gian và bộ nhớ tiêu tốn của từng giải thuật trên từng input, và phần giải thích kết quả.

1.2 Hai bài toán được chọn

Nhóm chọn hai bài toán:

- **Pipes** (`https://www.puzzle-pipes.com/`): xoay các đoạn ống để tạo thành một mạng liên thông, không rò rỉ và không có chu trình. Hai giải thuật: tìm kiếm theo chiều sâu **DFS** (mù) và **Greedy Best-First** (heuristic).
- **Light Up / Akari** (`https://www.puzzle-light-up.com/`, cùng hệ bài toán Logic Puzzles): đặt bóng đèn trên ô trắng sao cho mọi ô trắng được chiếu sáng, không bóng nào chiếu thẳng vào bóng khác và mọi tường số có đúng số bóng kề. Hai giải thuật đo mới: **DFS** (mù) và **Backtracking + MRV + CP** (tìm kiếm ràng buộc có heuristic). A* vẫn có trong ứng dụng để tối ưu số bóng.

Mỗi thực nghiệm dùng input theo cặp với cùng giới hạn tài nguyên cho các giải thuật được so sánh. Light Up đo mới trên 60 bàn với seed khác nhau; Pipes giữ bộ số liệu đã công bố trước đó.

1.3 Môi trường và công cụ

- **Ngôn ngữ**: Python 3 (yêu cầu bắt buộc của đề bài). Môi trường phát triển dùng Python 3.13, quản lý phụ thuộc bằng `uv`.
- **Đồ họa và demo**: `pygame-ce` cho cả hai game; `pygame-gui` bổ sung điều khiển cho giao diện Light Up.
- **Kiểm thử và chất lượng mã**: `unittest` cho cả hai package, `ruff` kiểm tra phong cách mã; chạy tự động bằng GitHub Actions.
- **Đo lường**: `time.perf_counter` cho thời gian, `tracemalloc` cho đỉnh bộ nhớ cấp phát của Python.
- **Biên soạn tài liệu**: LaTeX (tectonic) cho báo cáo và slide.

1.4 Tổ chức mã nguồn

Mã nguồn đặt trong một kho chung, mỗi bài toán một package riêng biệt:

```
Introduction-to-AI/
  main.py             Launcher chung: uv run main.py [pipes|lightup]
  pipes_problem/      Game Pipes, DFS/Greedy, UI, benchmark, test
  light_up_problem/   Light Up, DFS/CSP/A*, UI, benchmark, test
  docs/               Phan tich hoc thuat, bang chung benchmark (CSV + md)
  report/             Nguon LaTeX cua bao cao (main-vi, main-en)
  slides/             Nguon LaTeX cua slides (main-vi, main-en)
```

Cách chạy chi tiết nằm trong `README.md` của kho. Hai game chạy bằng `uv run main.py pipes` và `uv run main.py lightup`.

1.5 Quy trình tìm hiểu và hiện thực

Nhóm thực hiện theo cùng một quy trình cho cả hai bài toán:

1. **Tìm hiểu luật chơi**: chơi trực tiếp trên trang gốc của từng puzzle, ghi lại luật bất biến (điều kiện thắng, ràng buộc), tham chiếu trang `puzzle-pipes.com` / `puzzle-light-up.com`.
2. **Phát biểu bài toán tìm kiếm**: xác định trạng thái, trạng thái đầu, tác động, chuyển tiếp, kiểm tra đích và chi phí; chọn cách biểu diễn trạng thái hiệu quả (tiền tố định hướng cho Pipes, bitmask cho Light Up).
3. **Thiết kế giải thuật**: một giải thuật mù (DFS) và một giải thuật heuristic (Greedy / MRV kết hợp propagation), kèm cắt tỉa hợp lệ; viết giả mã trước khi code.
4. **Hiện thực và demo**: cài đặt game trực quan với pygame, có chế độ chạy từng bước (*step-by-step*) cho quá trình tìm kiếm và phát lại lời giải.
5. **Sinh input**: xây dựng bộ sinh puzzle theo seed xác định, kiểm chứng tồn tại nghiệm khi sinh; giải thích nguồn gốc từng input.
6. **Đo lường**: chạy benchmark tự động, ghi CSV thời gian – bộ nhớ – số node; lưu bằng chứng trong `docs/benchmarks/`.
7. **Phân tích**: giải thích số liệu, đối chiếu lý thuyết với thực nghiệm, ghi rõ hạn chế.

2 Bài toán 1: Pipes

2.1 Tìm hiểu đề bài và luật chơi

Nhóm tìm hiểu Pipes qua việc chơi trực tiếp trên https://www.puzzle-pipes.com/ và quan sát hành vi của game gốc. Sau quá trình chơi thử, nhóm tóm tắt luật như sau:

- Bàn chơi $m \times m$ gồm các ô ống. Mỗi ô có *hình dạng cố định* (ống thẳng, khuỷu, chữ T, chữ thập, đầu cụt) và một *định hướng* (hướng xoay) có thể thay đổi; xoay là tác động duy nhất của người chơi.

- Mỗi hình dạng có một tập cổng ở các phía Bắc–Đông–Nam–Tây. Xoay ô không đổi số cổng, chỉ đổi hướng.
- Mục tiêu: xoay sao cho **mọi** cổng mở đều gặp cổng mở tương ứng của ô kề (không cổng nào chìa ra ngoài biên, không cổng nào *dangling*), toàn bộ các ô nối thành **một** mạng liên thông và mạng **không chứa chu trình**.
- Nguồn nước (ô đánh dấu trên game gốc) chỉ để trực quan hóa: khi mạng hoàn chỉnh, nước chảy từ nguồn tới mọi ô. Đổi vị trí nguồn không làm đổi tập các bàn hợp lệ.

Nhìn theo phân loại của giáo trình, đây là bài toán *một agent, tất định, quan sát được toàn phần, rời rạc, tĩnh*. Đây là bài toán *thỏa mãn* (satisfaction): mọi bàn hợp lệ đều là lời giải, bài toán không yêu cầu tối ưu số lần xoay. Trò chơi gốc cũng không tính điểm theo số bước xoay.

2.2 Phát biểu bài toán tìm kiếm

Gọi $n = m^2$, đánh số các ô theo thứ tự hàng–cột (row-major). Ô i có hình dạng cố định và miền định hướng R_i gồm các xoay phân biệt của hình dạng đó. Nhóm phát biểu bài toán như sau:

Thành phần	Định nghĩa trong cài đặt
Input	Bàn chơi đã xáo trộn (hình dạng + định hướng lưu sẵn), kích thước, nguồn
Trạng thái	Tiền tố $s = (a_0, \ldots, a_{k-1})$ các định hướng đã gán
Trạng thái đầu	Tiền tố rỗng $()$, biết trước toàn bộ hình dạng
Tác động	Gán một định hướng hợp lệ biên cho ô k (ô kế tiếp theo row-major)
Chuyển tiếp	$s' = s + (a_k)$, chỉ nhận khi qua hết các kiểm tra khả thi
Kiểm tra đích	$k = n$ và bàn hoàn chỉnh thỏa mọi luật
Chi phí	1 mỗi phép gán; không giải thuật nào dùng chi phí để xếp hạng
Lời giải	Một bộ định hướng đầy đủ hợp lệ

Bảng 1: Phát biểu bài toán tìm kiếm cho Pipes.

Điểm cần nhấn: *một bước của giải thuật* là gán **tuyệt đối** một định hướng cho ô kế tiếp, khác với *một bước của người chơi* là xoay 90 độ. Một phép gán có thể tương ứng 0–3 lần xoay. Cả hai giải thuật dùng cùng mô hình gán, nên việc so sánh chúng là công bằng. Nhóm cũng tách bạch trạng thái tìm kiếm (tiền tố rỗng) với bàn xáo trộn lưu trong file input: bàn xáo trộn cung cấp miền R_i và thứ tự duyệt các xoay, còn tìm kiếm luôn bắt đầu từ tiền tố rỗng, không dựa vào đáp án của generator.

2.3 Không gian tìm kiếm và cắt tỉa dùng chung

Số cách gán đầy đủ (chưa xét biên) là $U = \prod_i |R_i| \le 4^n$: ô khuỷu/T/đầu cụt có 4 xoay phân biệt, ống thẳng có 2, ô chữ thập có 1. Với bàn 4×4 điển hình $U = 4^{16} \approx 4{,}29 \times 10^9$; với 7×7 cận trên là $4^{49} \approx 3{,}17 \times 10^{29}$. Cây gán có độ sâu n, hệ số phân nhánh tối đa 4, và **mỗi tiền tố có đúng một cha** (thứ tự gán cố định), nên không cần bảng closed để chống lặp trạng thái.

Trước khi tìm kiếm, các định hướng chìa cổng ra ngoài biên bị loại. Với mỗi ứng viên con, hàm `compatible()` dùng chung cho cả hai giải thuật thực hiện ba lớp cắt tỉa hợp lệ (sound):

1. **Khớp với láng giềng đã gán**: mỗi miền R_i chưa gán bị thu hẹp chỉ giữ định hướng khớp cổng của các ô đã gán (cả cổng có lẫn cổng *không* có). Miền nào rỗng thì chặn nhánh.

2. **Chặn chu trình**: nếu các ô đã gán tạo chu trình thì loại (dùng cấu trúc disjoint-set).

3. **Kết nối khả dĩ**: dựng đồ thị *lạc quan* giữa các ô — có cạnh nếu hai ô *có thể* mở cổng về phía nhau với một định hướng còn lại nào đó. Nếu đồ thị này không liên thông thì không thể có mạng hoàn chỉnh, chặn nhánh.

Tại gốc, kiểm tra nhanh $2(n-1)$ cổng: mạng cây trên n đỉnh có đúng $n-1$ cạnh $= 2(n-1)$ cổng; xoay không đổi số cổng nên tổng sai chứng tỏ vô nghiệm ngay từ đầu. Điều ngược lại không đúng (đủ cổng chưa chắc có nghiệm). Cả ba lớp cắt tỉa đều chỉ loại trạng thái *không thể* dẫn tới nghiệm nên không mất tính đầy đủ. DFS do đó là giải thuật mù về *thứ tự duyệt*, nhưng vẫn là DFS có cắt tỉa ràng buộc chứ không phải máy đếm xoay vét cạn.

2.4 Giải thuật tìm kiếm mù: Depth-First Search

DFS đi sâu theo nhánh con khả thi đầu tiên cho đến khi gặp đích hoặc bế tắc rồi quay lui. Thứ tự duyệt các xoay là thứ tự lưu sẵn của từng ô — hoàn toàn không dùng thông tin heuristic:

```
SEARCH_DFS(input):
  build boundary-valid domains R_i
  if root feasibility fails: return UNSOLVABLE
  return VISIT(())

VISIT(s):
  if budget exhausted: return LIMIT_REACHED
  count one expansion
  if GOAL(s): return SOLVED(s)
  for a in boundary-valid rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):           # 3 pruning layers
      r = VISIT(child)
      if r is SOLVED or LIMIT_REACHED: return r
  return UNSOLVABLE
```

Với không gian hữu hạn, cây có chu kỳ bị cắt bởi cấu trúc tiền tố, và cắt tỉa là hợp lệ nên DFS *đầy đủ* nếu đủ tài nguyên. Lời giải đầu tiên tìm được không đảm bảo tối ưu số phép gán. Độ phức tạp thời gian xấu nhất là hàm mũ theo n; về bộ nhớ, DFS giữ đường hiện hành và các nhánh chưa xét — cài đặt Python giữ bản sao tiền tố và khung generator cho mỗi mức, khoảng $O(n^2)$ trên một nhánh.

2.5 Giải thuật heuristic: Greedy Best-First

Greedy luôn pop khỏi hàng ưu tiên (min-heap) trạng thái có ít *lựa chọn còn lại* nhất:

$$h(s) = \sum_{i \text{ chưa gán}} \log_2 |D_i(s)|, \qquad \text{priority}(s) = (h(s),\ -\text{depth}(s),\ \text{insertion order})$$

với $D_i(s)$ là miền biên-hợp-lệ của ô i sau khi khớp với các láng giềng đã gán. Tổng là logarit của tích các miền còn lại: ví dụ miền $(1, 2, 4)$ cho $h = 0+1+2 = 3$, còn $(1, 1, 2)$ cho $h = 1$ — trạng thái B ít "tự do" hơn và được ưu tiên. Nhóm chọn heuristic này vì: (i) nó rẻ, chỉ cần miền đã tính trong bước cắt tỉa; (ii) mọi lời giải ở cùng độ sâu đều cần đúng n phép gán nên $n-k$ không phân

biệt được gì, còn tổng log-miền ưu tiên nhánh ít phân nhánh nhất. h ở đây *không admissible* — nó đo độ bất định còn lại, không phải cận dưới chi phí; và không dùng thành phần g.

```
SEARCH_GREEDY(input):
  build boundary-valid domains; root feasibility check
  push () into min-priority queue with priority 0
  while queue is not empty:
    if budget exhausted: return LIMIT_REACHED
    s = pop minimum-priority state;  count one expansion
    if GOAL(s): return SOLVED(s)
    for a in boundary-valid rotations of tile |s|:
      child = s + (a);  D = FEASIBLE(child)
      if D exists: push child with (h(child), -|child|, serial)
  return UNSOLVABLE
```

Trên không gian tiền tố hữu hạn, vô chu kỳ, việc giữ toàn bộ ứng viên khả thi làm Greedy đầy đủ nếu đủ tài nguyên; nhưng Greedy phải giữ cả frontier trong heap nên bộ nhớ có thể tăng hàm mũ.

2.6 Hiện thực và demo trực quan

Cài đặt nằm trong package `pipes_problem`: `model.py` (luật, validation, sinh bàn), `search.py` (hai giải thuật, cắt tỉa, đo thời gian/bộ nhớ), `app.py` (giao diện pygame), `worker.py` (process đo độc lập), `benchmark.py` (chạy batch). Giao diện demo:

- Chơi thủ công: click xoay (Shift để ngược chiều), right-click khóa ô, Undo/Reset; ống có nước hiển thị màu xanh chảy từ nguồn.
- **Demo tìm kiếm từng bước**: chọn DFS hoặc Greedy rồi *Run / Pause / Next / Speed*. Mỗi sự kiện hiển thị đúng loại: *expand* (thăm/pop tiền tố), *assign* (DFS chọn con), *queue* (Greedy push), *reject* (định hướng bị cắt tỉa — ô viền đỏ kèm lý do), *backtrack*. Nút *Next* cho phép phân biệt *mở rộng trạng thái* với *sinh successor*.
- *Show solution* phát lại riêng bộ gán cuối cùng; bàn trở về trạng thái đầu trước khi chạy. Kết quả *vô nghiệm* và *chạm giới hạn* hiển thị thành hai thông báo phân biệt.

2.7 Tạo input

Nhóm sinh input bằng generator của project (không chép tay từ web): 9 map mặc định gồm 3 kích thước $4\times4, 5\times5, 7\times7\times3$ seed 261–263, định dạng JSON gọn (`size`, `source`, `tiles`), tên file là định danh input. Ngoài ra có `maps/custom/cdai.json` — map được hiệu chỉnh tay thành *vô nghiệm* để kiểm tra thông báo và hành vi cạn không gian. Generator runtime cũng cho sinh map theo `--size`/`--seed`. Đây là input *mẫu của nhóm* (không phải map chính thức của trang web) và generator không lưu nghiệm vào file map.

2.8 Kết quả thực nghiệm: thời gian và bộ nhớ

Giao thức đo. Chạy trên máy Linux x86_64 (kernel 6.8), Python 3.13.14, pygame-ce 2.5.8. Cả hai giải thuật nhận *cùng* 9 map, cùng thứ tự gán row-major, cùng bộ cắt tỉa, giới hạn 30 giây và 200,000 lần mở rộng mỗi lượt. Mỗi cặp (map, giải thuật) chạy 5 lần: thời gian là solve headless

bằng `perf_counter` (không vẽ hình); bộ nhớ là đỉnh cấp phát Python đo bằng `tracemalloc` trong một replay riêng, chỉ gộp khi hai lần cho cùng status, cùng bàn kết quả và cùng số lần mở rộng. Bảng số liệu đầy đủ (90 lượt) lưu tại `docs/benchmarks/2026-10-25-pipes.csv`; các bảng dưới là trung vị (median) theo từng input.

Input	Giải thuật	Solved/lượt	Thời gian (ms)	Bộ nhớ (KiB)
pipes-4x4-261	DFS	5/5	3.380	26.836
	Greedy	5/5	2.866	11.695
pipes-4x4-262	DFS	5/5	3.047	27.078
	Greedy	5/5	3.386	12.359
pipes-4x4-263	DFS	5/5	3.149	26.984
	Greedy	5/5	2.741	12.500
pipes-5x5-261	DFS	5/5	5.352	53.203
	Greedy	5/5	7.094	19.500
pipes-5x5-262	DFS	5/5	5.906	53.344
	Greedy	5/5	5.882	19.219
pipes-5x5-263	DFS	5/5	4.564	53.297
	Greedy	5/5	5.712	19.219
pipes-7x7-261	DFS	5/5	23.247	145.852
	Greedy	5/5	27.049	27.594
pipes-7x7-262	DFS	5/5	21.938	145.930
	Greedy	5/5	26.669	27.070
pipes-7x7-263	DFS	5/5	40.785	145.852
	Greedy	5/5	38.246	29.094

Bảng 2: Pipes — thời gian và đỉnh bộ nhớ theo từng input (median 5 lượt, đo 25/10/2026). Cả 90/90 lượt đều *solved*. Nguồn: `docs/benchmarks/2026-10-25-pipes.csv`.

Kích thước	Giải thuật	Solved/lượt	Median time (ms)	Median memory (KiB)
4×4	DFS	15/15	3.149	26.984
	Greedy	15/15	3.097	12.359
5×5	DFS	15/15	5.261	53.297
	Greedy	15/15	6.157	19.219
7×7	DFS	15/15	24.469	145.852
	Greedy	15/15	31.695	27.594

Bảng 3: Pipes — tổng hợp theo kích thước (3 map $\times$ 5 lượt mỗi ô số liệu).

Giải thích kết quả.

- **Thời gian tăng nhanh theo kích thước** (3–5 ms ở 4×4 lên 22–41 ms ở 7×7): số biến tăng từ 16 lên 49 và chi phí kiểm tra khả thi mỗi successor tăng theo số láng giềng đã gán, đúng dự đoán hàm mũ về không gian tìm kiếm.
- **DFS nhanh hơn Greedy ở 5×5 và 2/3 map 7×7.** Trên các input dễ, cắt tỉa đã khiến nhánh đầu tiên của DFS gần như thẳng tới nghiệm; hàng ưu tiên của Greedy lúc đó chỉ tạo thêm

chi phí push/pop heap và tính h mà không đổi thứ tự duyệt đáng kể. Đây là minh chứng cho việc heuristic *không miễn phí*: chi phí cài đặt phải được bù bằng chất lượng xếp hạng.

- **Trên map khó nhất (pipes-7x7-263), Greedy nhanh hơn DFS** (38.2 so với 40.8 ms): map này buộc tìm kiếm phải lui vài nhánh thật; xếp hạng theo "ít lựa chọn còn lại" giúp nhánh gấp gáp (miền thu hẹp nhanh) được xét trước, tiết kiệm bớt phần dò thừa. Chênh lệch nhỏ (5%) nhưng xuất hiện ở đúng loại input mà heuristic có giá trị.
- **Bộ nhớ: Greedy thấp hơn DFS 4–5 lần** (12–29 KiB so với 27–146 KiB). Nghịch với suy đoán lý thuyết "frontier của Greedy có thể hàm mũ", số đo tracemalloc phản ánh *cấp phát Python thực tế của cài đặt*: DFS giữ bản sao tiền tố cho từng mức đệ quy và khung generator treo (mỗi mức một tuple copy), trong khi Greedy trên input dễ chỉ giữ vài chục ứng viên do cắt tỉa gạt gần hết. Trên input khó hơn nhiều, frontier của Greedy vẫn có nguy cơ phình hàm mũ — cần thí nghiệm bổ sung để khẳng định.
- **Cả 90 lượt đều solved** trong giới hạn; không có run nào chạm 30 giây hay 200k expansions trên bộ input này.

2.9 Hạn chế

Bộ input mặc định khá dễ với cắt tỉa hiện tại nên hai giải thuật chưa bị ép tới giới hạn; kết luận về "kẻ thắng" chỉ có giá trị trên loại input này. Nhóm chưa đo riêng đóng góp từng lớp cắt tỉa (ablation), chưa thêm đủ nhiều map vô nghiệm, và chưa thử heuristic MRV. Các hạn chế này được ghi nhận trong `docs/pipes-analysis.md` và là việc làm tiếp theo.

3 Bài toán 2: Light Up (Akari)

3.1 Quá trình tìm hiểu và luật chơi

Light Up, còn gọi là Akari, là trò chơi suy luận trên lưới do Nikoli phát triển năm 2001 [5]. Nhóm bắt đầu từ luật chính thức của Nikoli, đối chiếu cách tương tác trên Puzzle Light Up, rồi tìm hiểu các hướng giải và cách đánh giá độ khó trước khi xây dựng chương trình [5, 6]. McPhail chứng minh bài toán quyết định "một bàn Light Up có lời giải hay không" là NP-complete trong trường hợp tổng quát, qua phép quy dẫn từ Circuit-SAT [1]. Kết quả này là cơ sở để nhóm chọn tìm kiếm kết hợp cắt tỉa và suy luận ràng buộc.

Bàn gồm ô trắng và tường đen; một số tường mang số từ 0 đến 4. Theo luật Nikoli [5], một lời giải phải thỏa mãn:

- Bóng chỉ đặt trên ô trắng, chiếu theo hàng và cột đến tường hoặc biên; ô chứa bóng cũng được tính là sáng.
- Mọi ô trắng đều phải được chiếu sáng.
- Hai bóng không được chiếu vào nhau; có thể cùng hàng hoặc cột nếu có tường chắn.
- Tường số k có đúng k bóng kề trực tiếp theo bốn hướng, không tính đường chéo. Tường không số chỉ chặn ánh sáng.

Trang Puzzle Light Up [6] được tham khảo cho cách chọn kích thước, mức độ, nhấp trái đặt bóng và nhấp phải đánh dấu $\times$. Từ đó, nhóm bổ sung chức năng xem từng bước giải thuật và theo dõi các chỉ số tìm kiếm phục vụ bài học.

3.2 Mô hình hóa bài toán

Gọi P là tập ô trắng. Mỗi ô $i \in P$ có biến $x_i \in \{0, 1\}$, với $x_i = 1$ nghĩa là có bóng. Đặt $V(i)$ là các ô trắng nhìn thấy từ i, kể cả chính nó; L là một đoạn ô trắng liên tục theo hàng hoặc cột; $N(w)$ là các ô trắng kề tường số w. Ba nhóm ràng buộc tương ứng với luật chơi là:

$$\sum_{j \in V(i)} x_j \geq 1 \qquad \text{(mỗi ô trắng được chiếu)},$$

$$\sum_{j \in L} x_j \leq 1 \qquad \text{(không có hai bóng nhìn nhau)},$$

$$\sum_{j \in N(w)} x_j = k_w \qquad \text{(đúng số bóng quanh tường)}.$$

Trạng thái đầu chưa có bóng; trạng thái đích thỏa cả ba nhóm ràng buộc. Trong code, tập bóng, tập ô sáng và tập vị trí còn hợp lệ được lưu bằng bitmask để cập nhật nhanh. Với CSP, ô có bóng có miền $\{1\}$, ô bị cấm có miền $\{0\}$, ô chưa quyết định có miền $\{0, 1\}$. Hàm kiểm tra đích được tách khỏi giao diện để dùng chung cho solver, kiểm chứng bàn sinh và xác nhận kết quả thực nghiệm.

3.3 Thiết kế độ khó và sinh bàn chơi

Nhóm tham khảo nghiên cứu của Dekker [2], trong đó độ khó được đánh giá theo các kỹ thuật suy luận cần dùng, từ xử lý tường/ô tối đến mẫu suy luận và bước đoán. Để đơn giản hóa phạm vi bài tập, chương trình **không áp dụng thang đo này**: ba nhãn easy/normal/hard chỉ khác nhau ở **mật độ tường mục tiêu**. Theo `light_up_problem/generator/generate.py`, với bàn $n \times n$, số tường là $M = \text{round}(\rho n^2)$:

Mức	ρ	7×7	10×10	14×14	25×25
easy	16%	8	16	31	100
normal	10.5%	5	10	21	66
hard	5.5%	3	6	11	34

M tính cả tường số lẫn tường không số; bảng dùng đúng cách làm tròn của Python. Việc sinh bàn ở cả ba mức theo cùng quy trình:

1. Dùng seed chọn ngẫu nhiên M vị trí tường.
2. Liên tiếp chọn một ô trắng còn tối để đặt bóng, đến khi mọi ô trắng sáng. Tập bóng thu được là một lời giải mẫu (witness).
3. Suy ra số trên tường từ các bóng kề trong witness; giữ khoảng 25% tường không số và kiểm chứng lại witness trước khi trả bàn cho người chơi.

Witness không được cung cấp cho solver. Các chỉ số về mật độ gợi ý và độ phủ ràng buộc được lưu trong metadata để mô tả bàn, không dùng để gán nhãn độ khó. Do đó, nhãn hard không bảo đảm mọi bàn đều khó hơn easy theo thời gian giải; chương trình cũng chưa kiểm tra tính duy nhất của lời giải.

3.4 Lựa chọn và hiện thực giải thuật

DFS làm mốc so sánh. DFS chọn ô tối đầu tiên theo row-major, thử các vị trí hợp lệ có thể chiếu ô đó, và đi sâu ngay khi gặp ứng viên khả thi đầu tiên. Khi nhánh thất bại, solver quay lui để thử ứng viên kế tiếp. Kiểm tra khả thi loại trường hợp xung đột, tường không còn đủ ô kề hoặc ô tối không còn nguồn chiếu. Các nhánh anh em loại dần ứng viên đã thử để tránh lặp tập bóng theo thứ tự đặt khác nhau. Cài đặt dùng stack frame tường minh và liên kết cha để dựng lời giải, không phụ thuộc đệ quy Python.

CSP: Backtracking + MRV + Constraint Propagation. Nhóm tham khảo cách kết hợp suy luận bắt buộc với backtracking trong mục 4.1.1 và 4.1.3 của Pulles [3], cùng mã `tracksolver.py` trong repository Borroot/akari [4]. Các nguồn này giúp định hướng việc quản lý ô tối, ứng viên và phục hồi trạng thái. Trong chương trình hiện tại, hướng đó được triển khai bằng bitmask, propagation tại mỗi nhánh và **MRV động trên ràng buộc**; đây là các lựa chọn hiện thực được mô tả theo code.

Constraint propagation (CP) lặp đến khi không còn thay đổi:

- Tường đủ bóng: cấm các ô kề chưa quyết định. Nếu số bóng còn thiếu bằng số ô kề hợp lệ: buộc đặt bóng ở tất cả các ô đó.
- Ô tối chỉ còn một nguồn chiếu hợp lệ: buộc đặt bóng tại nguồn đó. Mỗi bóng mới đồng thời cấm đặt bóng khác trên đường chiếu sáng của nó.
- Loại các vị trí gây xung đột đường chéo khi chúng làm tường không còn đủ ô kề để đạt số bóng yêu cầu; gặp mâu thuẫn thì loại nhánh.

Sau CP, MRV (Minimum Remaining Values) ưu tiên ràng buộc ít lựa chọn nhất: tường còn u ô kề và thiếu r bóng có $\binom{u}{r}$ tổ hợp; ô tối có số lựa chọn bằng số nguồn chiếu còn hợp lệ. Hòa điểm ưu tiên tường. Các tổ hợp xung đột được loại, còn ứng viên được thử theo số ô tối chiếu thêm, từ nhiều đến ít.

```
BACKTRACK_MRV_CP(state):
  check budgets; propagate until no change
  if contradiction: return failure
  if goal: return solution
  choose constraint with fewest remaining choices
  for choice ordered by lighting gain:
    child = apply choice to current checkpoint
    result = BACKTRACK_MRV_CP(child)
    if solved or limited: return result
    restore parent checkpoint before next choice
  return failure
```

Giả mã mô tả luồng quay lui; code dùng stack frame tường minh. Checkpoint phục hồi cả bóng lẫn miền hợp lệ. Hàm `backtracking_mrv_cp` chạy propagation cả ở gốc, rồi tìm lời giải đầu tiên; DFS và CSP đều không bảo đảm tối ưu số bóng. A* vẫn có trong ứng dụng nhưng không thuộc phép so sánh này. Một node CSP có thể chứa nhiều bóng suy ra, nên số node giữa hai solver không phải lượng công việc tương đương; benchmark đo hiệu ứng kết hợp MRV, CP và thứ tự thử.

3.5 Demo và phương pháp thực nghiệm

Giao diện cho phép chơi tay hoặc chọn solver, Start Search, Pause/Resume và xem lại lịch sử bằng hai mũi tên ở 2/8/30 bước mỗi giây. CSP hiện từng suy luận, đánh dấu $\times$ ở ô cấm và phục hồi đầy đủ khi quay lui. Worker riêng giữ giao diện phản hồi. Visualizer bỏ timeout nhưng vẫn giới hạn node/bộ nhớ; benchmark không bật trace/animation và giữ đủ ba giới hạn.

Đợt đo ngày 31/10/2026 dùng 4 kích thước $\times$ 3 mức $\times$ 5 seed khác nhau, tức 60 bàn và 120 lượt DFS/CSP. Mỗi bàn chạy một lần mỗi solver; tất cả lượt chạy tuần tự trong process mới, đảo thứ tự solver ở các seed liên tiếp. Các dải seed dùng chung cho hai solver là:

Cỡ	easy	normal	hard
7	70097–70101	70194–70198	70291–70295
10	100097–100101	100194–100198	100291–100295
14	140097–140101	140194–140198	140291–140295
25	250097–250101	250194–250198	250291–250295

Giới hạn chung là **30 giây, 500.000 node mở rộng, 512 MiB bộ nhớ ước lượng**. Giữ riêng `TIMEOUT`, `NODE_LIMIT`, `MEMORY_LIMIT` và `UNSOLVABLE`; không tự nới ngân sách. UNSOLVABLE chỉ được kết luận khi chứng minh mâu thuẫn hoặc đã hết nhánh, không phải khi chạm giới hạn. Thời gian đo bằng `perf_counter` bên trong solver khi bật `tracemalloc`; không tính sinh bàn, khởi động process hay kiểm chứng bên ngoài. Bộ nhớ là đỉnh cấp phát Python, khác RSS và khác giá trị ước lượng dùng để dừng solver. Mọi kết quả SOLVED đều được kiểm chứng độc lập. Do kiểm tra giới hạn tại checkpoint, thời gian có thể vượt nhẹ 30 giây.

CSV, summary và manifest chứa đầy đủ seed, bàn JSON, hash cùng thông tin môi trường tại `docs/benchmarks/`, với tiền tố `2026-10-31-lightup-dfs-csp`. Đây là bộ dữ liệu được dùng cho các bảng dưới.

3.6 Kết quả đo

Giải thuật	SOLVED	TIMEOUT	NODE_LIMIT	MEMORY_LIMIT
DFS	52	8	0	0
CSP	60	0	0	0

Thời gian, bộ nhớ và node là median trên cả năm lượt thử, kể cả lượt bị giới hạn. Cột Bóng chỉ tính lượt SOLVED, không coi TIMEOUT là 0; số mẫu theo cột Giải được. Khi số lượt giải được khác nhau, các median bóng có thể dựa trên tập seed khác nhau. Summary có thêm median thời gian riêng cho lượt SOLVED.

Đánh giá. DFS giải được **52/60** bàn, CSP **60/60**; tám lượt TIMEOUT đều thuộc DFS. Không lượt nào chạm NODE_LIMIT hay MEMORY_LIMIT, nhưng cả hai giới hạn vẫn được bật. Trong 52 cặp cùng giải xong, CSP nhanh hơn ở 15 cặp; median $t_{DFS}/t_{CSP} = 0.73$, nên không thể kết luận CSP luôn nhanh hơn. Bàn nhỏ đã cắt tỉa tốt có thể lợi cho DFS do CP tốn thêm công việc mỗi node; CSP cải thiện khả năng giải các trường hợp rẽ nhánh lớn trong cùng ngân sách. Số bóng chỉ mô tả nghiệm tìm được, không chứng minh tối ưu. Số liệu DFS/A* cũ ngày 25/10 dùng generator trước đó được lưu riêng, không trộn vào bảng mới.

Năm seed mỗi cấu hình còn là mẫu nhỏ; độ khó theo mật độ tường và tính duy nhất nghiệm chưa được hiệu chỉnh/kiểm tra. Hướng tiếp theo là tăng mẫu, thêm bàn vô nghiệm, tách tác dụng CP/MRV và đo thời gian riêng khi tắt tracemalloc.

Cỡ	Độ khó	Thuật toán	Giải được	T/N/M	Giây	Đỉnh (KiB)	Node	Bóng
7	easy	DFS	5/5	0/0/0	0.0075	24.9	11	9
7	easy	CSP	5/5	0/0/0	0.0066	25.2	4	9
7	normal	DFS	5/5	0/0/0	0.0066	26.8	9	7
7	normal	CSP	5/5	0/0/0	0.0103	29.0	6	7
7	hard	DFS	5/5	0/0/0	0.0074	27.3	8	7
7	hard	CSP	5/5	0/0/0	0.0107	30.5	6	6
10	easy	DFS	5/5	0/0/0	0.0229	55.9	27	15
10	easy	CSP	5/5	0/0/0	0.0288	56.2	9	14
10	normal	DFS	5/5	0/0/0	0.0218	60.0	15	12
10	normal	CSP	5/5	0/0/0	0.0309	61.8	8	11
10	hard	DFS	5/5	0/0/0	0.0229	62.4	12	11
10	hard	CSP	5/5	0/0/0	0.0314	67.8	10	10
14	easy	DFS	5/5	0/0/0	0.2330	135.3	261	28
14	easy	CSP	5/5	0/0/0	0.0801	109.1	12	25
14	normal	DFS	5/5	0/0/0	0.0724	117.3	34	22
14	normal	CSP	5/5	0/0/0	0.1030	123.5	17	19
14	hard	DFS	5/5	0/0/0	0.0832	126.1	22	19
14	hard	CSP	5/5	0/0/0	0.1168	134.6	15	16
25	easy	DFS	2/5	3/0/0	30.0001	2237.0	9 328	73
25	easy	CSP	5/5	0/0/0	0.9558	562.1	31	68
25	normal	DFS	1/5	4/0/0	30.0004	3583.3	13 886	58
25	normal	CSP	5/5	0/0/0	1.0274	696.9	33	52
25	hard	DFS	4/5	1/0/0	0.7835	870.3	46	40.5
25	hard	CSP	5/5	0/0/0	1.0421	868.0	34	35

Bảng 4: Median trên năm seed khác nhau mỗi cấu hình. T/N/M lần lượt đếm TIMEOUT, NODE_LIMIT, MEMORY_LIMIT. Thời gian gồm cả lượt bị giới hạn, không phải thời gian giải xong. Cột Bóng là median số bóng chỉ trên các lượt SOLVED; số mẫu theo cột Giải được.

4 Kết luận

Repository có demo tìm kiếm trực quan cho cả hai bài toán. Pipes so DFS với Greedy Best-First; thực nghiệm Light Up mới so DFS với Backtracking + MRV + CP. A* vẫn có trong Light Up nhưng số liệu lịch sử không được trộn với generator hiện tại và thực nghiệm năm seed khác nhau.

- **Heuristic có chi phí.** Propagation giảm tìm kiếm nhưng tốn thêm công việc mỗi node. Trên bàn nhỏ cắt tỉa tốt, DFS có thể nhanh hơn; CSP cải thiện khả năng giải trong ngân sách ở các trường hợp rẽ nhánh lớn.
- **So sánh theo cặp.** Năm seed mỗi cấu hình lấy mẫu nhiều input, không phải lặp năm lần cùng một bàn. Hai giải thuật nhận cùng map và cùng giới hạn thời gian/node/bộ nhớ ước lượng.
- **Giới hạn không phải vô nghiệm.** Ghi riêng TIMEOUT, NODE_LIMIT, MEMORY_LIMIT. Thời lượng lượt bị dừng không ước lượng thời gian giải xong. Xem bảng kết quả Light Up và CSV gốc.
- **Định nghĩa phép đo.** Đỉnh cấp phát Python không phải RSS. Giới hạn bộ nhớ dùng giá trị ước lượng của solver; node CSP và DFS cũng không đại diện cùng lượng công việc.

Hướng tiếp theo: tăng số seed, thêm bàn vô nghiệm, ablation CP/MRV/cắt tỉa, kiểm tra duy nhất và đo thời gian riêng khi tắt tracemalloc.

Tài liệu

[1] B. McPhail. *Light Up is NP-complete*, 2005. https://www.mountainvistasoft.com/docs/lightup-is-np-complete.pdf

[2] V. Dekker. *A Difficulty Measure for Light Up Puzzles*. Luận văn cử nhân, Leiden University, 2015. https://theses.liacs.nl/pdf/2014-2015Victor-Dekker.pdf

[3] B. Pulles. *Analysis of Akari*. Luận văn cử nhân, Radboud University, 2021; mục 4.1.1 và 4.1.3. https://www.cs.ru.nl/bachelors-theses/2021/Bram_Pulles___1015194___Analysis_of_Akari.pdf

[4] Borroot. *akari*: mã nguồn bộ giải, đặc biệt `src/tracksolver.py`. https://github.com/borroot/akari

[5] Nikoli. *Akari*: luật chơi chính thức. https://www.nikoli.co.jp/en/puzzles/akari/

[6] Puzzle Light Up. Trò chơi trực tuyến và giao diện tham khảo. https://www.puzzle-light-up.com/

[7] Puzzle Pipes. https://www.puzzle-pipes.com/

[8] S. J. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Pearson, 2021 (chương 3: Tìm kiếm trong không gian trạng thái; chương 6: Ràng buộc).

[9] Python 3 documentation: `time`, `tracemalloc`, `multiprocessing`, `heapq`, `unittest`. https://docs.python.org/3/

[10] pygame-ce documentation. https://pyga.me/docs/ pygame_gui documentation. https://pygame-gui.readthedocs.io/